

Contents



Miscellaneous (M)

intro	Introduction to miscellaneous features and files.
aliases, aliases.hash,	
maliasies, maliasies.hash,	
faliasies	Micnet alias files.
aliashash	Micnet alias hash table generator
ascii	Map of the ASCII character set.
badtrack	Creates bad track table for the fixed disk.
boot	XENIX boot program.
cd	Disk cartridge
check	Verifies the storage capacity of a tape cartridge.
clock	Accesses the real-time clock file.
cmos	Displays/sets values in the CMOS data base.
console	Sets the standard terminal system.
daemon.mn	Micnet mailer daemon
default	Default program information directory.
environ	The user environment.
fd	Floppy disk drives.
fdisk	Creates the disk partition table on the fixed disks.
getty	Sets terminal mode.
group	Format of the group file.
hd	Fixed disk drives.
init	Process control initialization.
installidd	Installs installable device drivers.
ld	Invokes the link editor.
login	Gives access to the system.
lp	Printer device interface.
mem, kmem	Memory image file.
messages	Description of system console messages.
micnet	The Micnet default commands file.
mt	Streaming tape drive.
null	The null file.
passwd	The password file.
profile	Sets up an environment at login time.
serial	Serial port interface.
systemid	The Micnet system identification file.
term	Conventional names for terminals.
termcap	Terminal capability database.



Miscellaneous (M)

terminals

List of supported terminals.

termio

General terminal interface.

top, top.next

The Micnet topology files.

tty

Special terminal interface.

ttys

Login terminals file.

Name

intro – Introduction to miscellaneous features and files.

Description

This section contains miscellaneous information for maintaining the entire system, including descriptions of files, devices, tables, and programs.



Name

aliases, aliases.hash, maliases, maliases.hash, faliases – Micnet alias files.

Description

These files contain the alias definitions for a Micnet network. *Aliases* are short names or abbreviations that may be used in the *mail* command to refer to specific machines or users in a network. Aliases allow a complex combination of site, machine, and user names to be represented by a single name.

The **aliases**, **maliases**, and **faliases** files each define a different type of alias. The **aliases** file defines the standard aliases which are names for specific systems and users and, in some cases, for commands. The **maliases** file defines machine aliases, names, and paths for specific systems. The **faliases** file defines forwarding aliases which are temporary names for forwarding mail intended for one system or user to another.

The **aliases.hash** file is the hashed version of the **aliases** file created by the *aliashash* command. The file is used by the *mail* command to resolve all standard aliases and is identical to the **aliases** file except for a hash table at the beginning of the file. The hash table allows for more efficient access to the entries in the file. The **aliases** file need only be present to generate the **aliases.hash** file. The **aliases** file is not required to run the network.

The **maliases.hash** file is the hashed version of the **maliases** file. It is an optional file created by executing the following command:

```
/usr/lib/mail/aliashash /usr/lib/mail/maliases
```

If the **maliases.hash** file is created, **maliases** is no longer necessary to run the network. If the number of machines in the network is large, and particularly if several types of networks are in use, it is recommended that the **maliases** file be hashed. In such a network, the configuration is no longer homogeneous, aliases are likely to be fairly complex, and machine aliases are likely to differ between machines. The use of machine aliases allows the standard alias file to be identical on all machines in the network. In such an environment, *netutil* can only generate network files that can be used as a starting point. The rest of the network maintenance should be done manually with a text editor.

Each file contains zero or more lines. If hashing is to be performed, at least one alias is required. Each line lists the alias and its meaning. The alias meaning can have site, machine, and user login names and other aliases. Its exact composition depends on the type of alias. A colon (:) separating the alias and meaning is required.

In the **aliases** file, a line can have the forms:

```
alias:[[site!]machine:]user],[[site!]machine:]user...
```

```
alias:[[site!]machine:]command-pipeline
```

```
alias:error-message
```

Site and *machine* are the site and machine names of the system to which the user belongs or on which the specified command is to be executed. The site and machine names must end with an exclamation mark (!) or colon (:) respectively, and must be defined in a `systemid` file. A machine alias may be used in place of a site and machine name if it is followed by a question mark.

User is a user login name or another alias. User names in a list must be separated by commas. A NEWLINE may immediately follow a comma. Spaces and TABs are allowed, but only immediately before or after a comma or NEWLINE.

Command-pipeline is any valid command (with necessary arguments) preceded by a pipe symbol (|) and enclosed in double quotation marks. Spaces may separate the command and arguments, but there must be no space between the first double quotation mark and the pipe symbol.

Error-message is any sequence of letters, numbers, and punctuation marks (except a double quotation mark) preceded by a number sign (#) and enclosed in double quotation marks.

In the `faliases` file, each line can have the same form as lines in the `aliases` file except that no more than one user name can be given for any one alias. To prevent alias expansion on a remote machine, the meaning should be escaped with “\”, as in:

```
foo: mach?\foo
```

Failure to do the escape may result in an infinite forwarding loop. If this happens and the loop does not invoke a `uucp` connection, looping will be detected, and the mail will be returned to the sender.

The `alias.hash` file has already been searched at this point. If there is no explicit machine given as part of the meaning, the recipient will be assumed to be local. After forward aliases are searched, machine aliases are searched as necessary.

In the `maliases` file, a line has the form:

```
alias:[[site!]machine:]...
```

Site and *machine* are the site and machine names for a specific network and system. Multiple site and machine names direct messages along the specified path of systems. If no site or machine name is given, the alias is ignored.

Before the *mail* program sends a message, it searches the **aliases.hash**, **faliases**, and **malias** files to see if any of the names given with the command are aliases. Each file is searched in turn (**aliases.hash**, **faliases**, then **malias**) and if a match is found, the alias is replaced with its meaning. If no match is found, the name is assumed to be the valid login name of a user on that machine. The search in the **aliases.hash** file continues until all aliases have been replaced, so it is possible for several replacements to occur for a single name. Alias loops are now detected. If a loop exists, any recipients involved in the alias loop will be dropped from the mail recipient list, and an error message will be generated. The **faliases** file is searched once, from beginning to end, even if it is empty. The **malias** file is searched only if the alias contains a machine alias.

When an alias is a user or a list of users, the *mail* command sends the message to each user in the list. When it is a command-pipeline, the *mail* command starts execution of the command on the specified machine and sends the message as input. When the alias is an error-message, the *mail* command ignores the message and instead displays the alias and its meaning at the standard error.

In all files, any line beginning with a number sign (#) is considered a comment and is ignored.

As a special feature, any alias that contains a site name as the first component of its meaning is automatically prepended with the machine alias *uucp?*. This alias may be explicitly defined in the **malias** file to help direct mail between networks to the system performing the **uucp** link.

Directives

Though alias directives are never included in an alias expansion, they can be used to restrict the expansion to a class of users, forward the unexpanded alias to another machine, or produce error messages. An **aliases** file may include directives of the form:

```
testalias: $xalaska, mikem, georger, terih
```

```
sams: "$e ambiguous, use samst or samsm"
```

Fields on the right-hand side of an alias (after the colon) that begin with a dollar sign (\$) character, are alias directives. Fields containing any spaces or tabs must be enclosed in quotation marks. The directive must precede all normal right-hand fields as shown in the example above. The character following the dollar sign (\$) specifies the directive type:

```
$n <real name or description>
```

```
$x <machine>
```

\$e <error message>

\$p <permissions>

\$r <restrictions>

None of the above directives are currently supported by **faliases**. Only the \$e is supported by **mailaliases** and **mailaliases.hash**. Unrecognized directives do not create error messages and are treated as if they do not exist. The above directives are described in detail as follows:

\$n

For a user alias, this field should contain the full real name of the user associated with the alias. For a group alias, a description of the group should be given.

\$x

Causes the alias to be forwarded, unexpanded, to the machine specified in this field. White space is only allowed immediately following the \$x. Since machine aliases will be searched, the appropriate machine alias must exist in the **mailaliases** file.

\$e

This field contains an error message to be printed. The left side of the alias will be removed from the list of users to be aliased. An alternate form of \$e is #.

\$p

This field contains the * character or a string of upper and lowercase alphabetic characters. Each character indicates that the user on the left-hand side of the alias belongs to a special "class" of users. The * character implies membership in all such classes.

\$r

This field contains a string of upper and lower case alphabetic characters, each character indicating a "class" of users to be granted expansion permission. The absence of a \$r field means that any user can expand the alias. If the \$r field exists, expansion is only allowed if:

- 1) The user requesting expansion has a \$p field containing one or more of the characters found in the \$r field, or if:
- 2) The user has a \$p field and it contains a "*", or if:
- 3) The real user ID is 0 (super-user).

If expansion is not allowed, no error messages result; the alias in question is treated as if it were not present.

To send serious mail delivery problems to the super-user, the following alias could be used:

network: “\$n the network mail recipient,” root

To forward a group alias called *testalias* to a machine called *alaska* and expand it there, the following alias may be used:

testalias: \$xalaska, mikem, georger, terih

Files

/usr/lib/mail/aliases

/usr/lib/mail/aliases.hash

/usr/lib/mail/maliases

/usr/lib/mail/faliases

/usr/lib/mail/maliases.hash

See Also

aliashash(M), netutil(C), systemid(M), top(M), uucp(C)



Name

aliashash – Micnet alias hash table generator.

Syntax

```
aliashash [ -v ] [ -o output-file ] [ input-file ]
```

Description

The *aliashash* command reads the *input-file* and generates an *output-file* containing a hash table of alias definitions for a Micnet network. The *input-file* must name a file containing alias definitions in the form described for the **aliases** file (see *aliases*(M)). If the **-o** option is not used to specify an *output-file*, the command creates a file with the same name as the *input-file* but with **.hash** appended to it. If no *input-file* is given, the command reads the file named **/usr/lib/mail/aliases** and creates the file named **/usr/lib/mail/aliases.hash**.

If invoked with the **-v** option, the command lists information about the hash table.

The *output-file* will contain both the alias definitions given in the *input-file* and the new hash table. The hash table appears at the beginning of the file and is separated from the alias definitions by a blank line. The hash table has three or more lines. The first line is:

```
#<hash>
```

The second line has 4 entries: the bytes per table entry, the maximum number of items per hash value, the number of entries in the table, and the offset (in bytes) from the beginning of the file to the beginning of the alias definitions.

The next lines (up to the end of the hash table) contain the hash table entries. Each line has 8 entries (separated by spaces) and each entry has 2 fields. The first field (1 byte) is a checksum (represented as a printable character); the second field is a pointer (in bytes) to the alias definition. The pointer is represented as a hexadecimal number with leading blanks if necessary and is always relative to the start of the definitions.

The *aliashash* command is normally invoked by the **install** option of the *netutil* command. If the alias definitions of a network must be changed, the definitions in the **aliases** file should be changed and a new **aliases.hash** file created using the *aliashash* command. The new **aliases.hash** file must then be copied to all other computers in the network.

Files

/usr/lib/mail/aliashash

/usr/lib/mail/aliases

/usr/lib/mail/aliases.hash

/usr/lib/mail/maliases.hash

See Also

aliases(M), netutil(C)

Warning

Do not use the *aliashash* command to create the **aliases.hash** file while the network is running. If necessary, create a temporary output file, **aliases.hash-**, using the **-o** option. Then type:

```
m.v aliases.hash- aliases.hash
```

This will prevent disruption of the network.

Name

ascii – Map of the ASCII character set.

Description

Ascii is a map of the ASCII character set. It lists both octal and hexadecimal equivalents of each character. It contains:

Octal ASCII

000 nul	001 soh	002 stx	003 etx	004 eot	005 enq	006 ack	007 bel
010 bs	011 ht	012 nl	013 vt	014 np	015 cr	016 so	017 si
020 dle	021 dc1	022 dc2	023 dc3	024 dc4	025 nak	026 syn	027 etb
030 can	031 em	032 sub	033 esc	034 fs	035 gs	036 rs	037 us
040 sp	041 !	042 "	043 #	044 \$	045 %	046 &	047 ^
050 (	051)	052 *	053 +	054 ,	055 -	056 .	057 /
060 0	061 1	062 2	063 3	064 4	065 5	066 6	067 7
070 8	071 9	072 :	073 ;	074 <	075 =	076 >	077 ?
100 @	101 A	102 B	103 C	104 D	105 E	106 F	107 G
110 H	111 I	112 J	113 K	114 L	115 M	116 N	117 O
120 P	121 Q	122 R	123 S	124 T	125 U	126 V	127 W
130 X	131 Y	132 Z	133 [	134 \	135]	136 ^	137 _
140 `	141 a	142 b	143 c	144 d	145 e	146 f	147 g
150 h	151 i	152 j	153 k	154 l	155 m	156 n	157 o
160 p	161 q	162 r	163 s	164 t	165 u	166 v	167 w
170 x	171 y	172 z	173 {	174	175 }	176 ~	177 del

Hexadecimal ASCII

00 nul	01 soh	02 stx	03 etx	04 eot	05 enq	06 ack	07 bel
08 bs	09 ht	0a nl	0b vt	0c np	0d cr	0e so	0f si
10 dle	11 dc1	12 dc2	13 dc3	14 dc4	15 nak	16 syn	17 etb
18 can	19 em	1a sub	1b esc	1c fs	1d gs	1e rs	1f us
20 sp	21 !	22 "	23 #	24 \$	25 %	26 &	27 ^
28 (	29)	2a *	2b +	2c ,	2d -	2e .	2f /
30 0	31 1	32 2	33 3	34 4	35 5	36 6	37 7
38 8	39 9	3a :	3b ;	3c <	3d =	3e >	3f ?
40 @	41 A	42 B	43 C	44 D	45 E	46 F	47 G
48 H	49 I	4a J	4b K	4c L	4d M	4e N	4f O
50 P	51 Q	52 R	53 S	54 T	55 U	56 V	57 W
58 X	59 Y	5a Z	5b [	5c \	5d]	5e ^	5f _
60 `	61 a	62 b	63 c	64 d	65 e	66 f	67 g
68 h	69 i	6a j	6b k	6c l	6d m	6e n	6f o
70 p	71 q	72 r	73 s	74 t	75 u	76 v	77 w
78 x	79 y	7a z	7b {	7c	7d }	7e ~	7f del

Files

/usr/pub/ascii



Name

badtrack, *badtrack1*, *badtrack2* – Creates a bad track table for the fixed disk

Syntax

```
device /etc/badtrack  
device /etc/badtrack1  
device /etc/badtrack2
```

Description

Badtrack is normally used during installation to find flaws on the fixed disk media. It records these flaws plus any additional ones manually entered, so that XENIX will not access these locations on the fixed disk. *Badtrack* does not block out flawed areas for other operating systems partitions, such as DOS.

Badtrack checks all fixed disks connected to the system, *badtrack1* checks the primary fixed disk only, and *badtrack2* checks the secondary fixed disk only.

Device is “fd” for floppy disk.

To use *badtrack* during the installation process, start up your system using the XENIX installation diskette. At the XENIX Boot prompt, type this command:

```
fd /etc/badtrack
```

and press RETURN:

Badtrack displays the drive number and the type of fixed disk it is checking. As *badtrack* checks each cylinder, it displays the cylinder's number. When complete, *badtrack* displays all bad tracks found and the following prompt:

```
Enter additional bad tracks for drive x  
Enter cylinder [0-xxx]  
(press Enter to terminate):
```

A Media Error Map might have come with your fixed disk. If it shows any additional flaws, or if you know of any other bad tracks, enter them now. It is very important that all questionable tracks are mapped out so that XENIX will not attempt to use them. Using flawed tracks can result in fixed disk errors and any information stored on that track can be lost. Enter the cylinder number of the bad track. The screen shows:

```
Enter track [0-x]
```

Enter the track number of the bad track.

Continue until you have entered all known bad tracks. After entering the last bad track, press RETURN to quit.

Badtrack now displays the bad track map. If you have a second fixed disk, *badtrack* repeats the procedure for that drive.

When *badtrack* completes execution, the screen shows the XENIX Boot prompt.

Notes

You can enter up to 244 bad tracks into the badtrack table.

If you add a secondary fixed disk to your system or replace a secondary fixed disk, we recommend that you run *badtrack2*. *Badtrack2* does not check the primary fixed disk and therefore does not alter the bad track map of the primary fixed disk. In the same note, if you replace the primary fixed disk and do not want to alter the bad track map of the secondary fixed disk, use *badtrack1* to check the disk.

The bad track map is located at the beginning of each fixed disk starting at offset 0x200, immediately following the partition table.

Altering the bad track map of a fixed disk which has XENIX partitions on it will result in unpredictable disk access by the fixed disk driver. Run *badtrack* only before XENIX partitions are created.

The *badtrack* command uses a non-destructive verification technique (read-only) to determine badtracks. Therefore, it is strongly recommended that you enter all known defective tracks into the bad track table.

Diagnostics

too many bad tracks

Badtrack detected more than 244 bad tracks.

Can't mark the badtrack table bad

Cylinder 0, Track 0 is bad, or Track 0, Sector 2 cannot be written to.

disk read failure

Badtrack could not successfully read from the fixed disk.

disk write failure

Badtrack could not successfully write to the fixed disk.

disk restore failure

Badtrack could not write to either Cylinder 0, Track 0 or Track 0, Sector 2.

Name

boot – XENIX boot program.

Description

Boot is an interactive program used to load and execute stand-alone XENIX programs. It is used primarily for loading and executing the XENIX kernel, but can be used for any other programs that have been linked for stand-alone execution. *Boot* is a required part of the XENIX operating system and must be present in the root directory of the root file system to ensure successful loading of the XENIX kernel.

Boot is invoked by the system each time the computer is started. A bootstrap loader, loaded by the startup ROM, carries out the invocation. A floppy disk version of *boot* is invoked if the upper floppy drive (drive 0) contains a floppy disk and block 0 of that floppy disk contains a valid bootstrap program. A fixed disk version of *boot* is invoked if the floppy drive is empty and block 0 of the active partition of the fixed disk contains a valid bootstrap program.

When first invoked (or after termination of standalone programs), *boot* prompts for the location of a program to load by displaying the message:

```
SYSV  XENIX Boot
```

```
Enter:  hd  program
        fd  program
        dos
        cf  [-c config_file] [device  program]
```

```
Press Enter for default:  hd /xenix
:
```

To specify the location of a program, a *device* and *pathname* must be given. The *device* can be either **fd**, for the floppy disk, or **hd**, for the active partition of the fixed disk. The *pathname* must be the full pathname of the file containing the stand-alone program. For example, to load **/xenix** from the floppy disk, type:

```
fd /xenix
```

Since *boot* is most often used to load the XENIX kernel from the fixed disk, the default value “**hd /xenix**” is provided.

The **dos** option can be used if you have partitioned your fixed disk to allow both XENIX and MS-DOS to be used. Entering the **dos** option will cause the *boot* program to search for an MS-DOS partition and invoke DOS.

The **cf** option is used if you want the boot loader to build a kernel image which incorporates specified installable device drivers. This option is explained in detail in Chapter 10 of the *XENIX Operations Guide*.

If you do not choose an option within 50 seconds, *boot* automatically loads the default (*hd /xenix*) kernel and proceeds with normal startup.

Diagnostics

For an error, *boot* displays an error message, then returns to its prompt. The following is a list of the most common messages:

bad magic number

The given file is not an executable program.

bad runtime environment

The given program is not a stand-alone program. The program must be a small or middle model program, and the absolute flag of the program header must be set.

pathname not found

The supplied pathname does not correspond to an existing file.

Stage 1 boot failure

The bootstrap loader cannot find or read *boot*. You must restart the computer and supply a file system disk with *boot* in the **root** directory.

Unknown device: *device*

Only **fd** and **hd** can be used for device names.

Link failed

The loader was unable to correctly load and link the specified kernel with the required installable device drivers. You must reboot with another version of the kernel.

Files

/boot

See Also

hd(M), ld(CP)

Notes

Boot cannot be used to load programs that have not been linked for stand-alone execution. To create stand-alone programs, the **-A** option of the XENIX linker must be used (see *ld*(CP)).

Although stand-alone programs can operate in real or protected mode, they must not be large or huge model programs. Programs in real mode can use the input/output routines of the startup ROM. Programs that wish to return control to *boot* must be linked with the `/usr/sys/boot/lib_stand` library.



Name

cd – disk cartridge

Description

The Disk Cartridge System is an optional unit containing one or two drives which accept flexible media enclosed in a rigid cartridge. Each disk cartridge must be properly formatted and initialized with the **format(M)** utility.

The binary representation of the minor device number is encoded *00ddppp*, where *dd* is a physical drive number (zero or one), and *ppp* is a disk partition (subsection) within a physical unit. The descriptions of the disk partitions are:

Disk	Description
0	entire disk, except boot track
1	file system on disk, if any, else invalid
2	alternate area on disk, if any, else invalid
3	boot track
4	reserved
5	reserved
6	reserved
7	reserved

The alternate area on a disk is defined to begin at the end of the file system (if any) on the disk and include the rest of the disk. The names of the boot-track partitions conventionally have the letters “bt” inserted before the drive number. The boot track reserves one physical track of the disk. The remainder of the space is available.

The block files access the disk via the system’s normal buffering mechanism and may be read and written without regard to physical disk records.

A “raw” interface provides for direct transmission between the disk and the user’s read or write buffer. A single read or write call results in exactly one I/O operation and therefore raw I/O is considerably more efficient when many words are transmitted. The names of the raw files conventionally begin with the prefix ‘r.’ In raw I/O, the buffer must begin on a word boundary, the disk address must be an exact multiple of 256, and the number of bytes in the I/O request must be a multiple of 256.

When any *dev* on the Disk Cartridge System is open, the cartridge cannot be removed from the drive. After the last close on the drive, the media continues to spin for 30 seconds and then stops. Once the media stops, the green indicator goes out and you can remove the media. During the 30 second interval, the media-release button on the drive stops the media immediately and makes it available for removal.

Cartridges with less than their rated amount of space will not format and will not open successfully. (A new cartridge has about 30% more space than the rated amount, but only the rated amount can be accessed.)

Startup Diagnostics

When this driver is installed, it displays a status line on the console when XENIX is booted. In addition to the version code for the driver, a series of checks are made on the hardware to insure that it is ready for use. If the hardware is working properly, then “ABCD–OK” is displayed. Each letter indicates that a test has been successfully completed.

Code	Meaning
A	The SCSI (Small Computer System Interface) Bus Adapter was located.
B	The SCSI Bus was reset and selected successfully. (This indicates cable continuity.)
C	The controller in the DCS cabinet is responding.
D	The master drive performed a test command successfully.

If any of these fail, the driver aborts all future open attempts with ENXIO, and the status message displays “Not Installed” or “ERROR – Device Disabled.”

Files

Block I/O:

```
/dev/cd0
/dev/cd1
/dev/cdbt0
/dev/cdbt1
```

Character I/O

```
/dev/rcd0
/dev/rcd1
/dev/rcdbt0
/dev/rcdbt1
```

See Also

backup(C), format(C), messages(M), mkfs(C), mount(C), tar(C)

Notes

In programs that are likely to access raw devices, *read*, *write* and *lseek*(S) should always deal in 256-byte multiples.

Never write to the boot devs. It contains information on the media, and the system can not open the cartridge if this data is missing or altered.

The **format(C)** utility formats cartridges with a physical interleave of 4:1.

When placing file systems on cartridges, use these parameters for **mkfs(C)** with a 10 megabyte cartridge:

```
mkfs /dev/rcdx 9760 3 32
```

For a 20 megabyte cartridge use:

```
mkfs /dev/rcdx 20896 3 32
```

These block counts should be used with **tar (C)** or **backup (C)** to indicate when volume changes should be requested.

The cartridge disk driver in the kernel expects that its device will share DMA channel 3 with the streaming tape device. It is mandatory that the jumpers on the cartridge disk controller board are positioned correctly for this driver to work properly. The distributed kernel assumes the cartridge disk device uses interrupt request line 5.



Name

check – Verifies the storage capacity of a tape cartridge.

Description

The *check* utility writes a test pattern to a tape cartridge (which is used in the optional Cartridge Tape Drive) to determine how much of the recording area of the tape is still usable.

As a tape ages, sections of it can become incapable of holding data. When data is written to the tape it is verified as it is written and if a bad area is detected, the drive writes another copy of the current block on the next usable section of tape. (This is a transparent operation.) Because of this, the amount of data a tape holds can decrease over time. However, the *sysadmin* utility expects each tape to hold 32 megabytes of data. This assumption is required since *dump* and *tar* must be told how many blocks can be written to the tape.

If the tape cannot hold enough data, the *tar* or *dump* utility aborts with a tape write error. To help prevent this from happening, use the *check* utility to certify that the tape can hold at least 32 megabytes of data.

The *check* utility makes its determination by writing over the entire tape and determining how many blocks can be written to the tape. If the tape can not hold 32 megabytes, *check* reports that “unusable.” If 32 megabytes or more is available, *check* declares the tape “okay.” In either case, *check* reports how much data the tape held.

See Also

mt(M), *dump*(C), *tar*(C), *sysadmin*(C)

Notes

The *check* utility is destructive and erases everything on the tape that is tested.

The test takes at least 10 minutes per tape.



Name

chswap – Changes the swap space size of a XENIX kernel.

Syntax

chswap *swapsize kernel*

Description

The *chswap* utility lets the super-user modify the swap space size in the designated kernel. The utility is used in the **hdinit** installation program to let the system administrator change the swap area size to the desired size. The *swapsize* argument is the size of the desired swap area in 1024-byte blocks. The *kernel* argument is the name of the XENIX kernel that should be modified.

Example

If you are using a larger swap area size than the default 3000 Kbyte blocks, and you are using installable device drivers, you need to run *chswap* on */lib/sys/xenix*. For example, if you installed your system with 6000 Kbytes of swap area, and you will be using installable device drivers, enter the following command:

```
/etc/chswap 6000 /lib/sys/xenix
```


Name

clock – Accesses the real-time clock file

Description

Use the *clock* file to access your computer's battery-powered real-time clock. You can get the current time by reading the file or change the time by writing to the file. The time is stored in the following format:

MMddhhmmyy

where: MM is the month
dd is the day
hh is the hour (24-hour time)
mm is the minute
yy is the last two digits of the year

For example:

0319153385

is March 19, 1985 at 3:33 p.m.

File

/dev/clock

See Also

setclock(C)



Name

`cmos` – Displays/sets values in the CMOS data base.

Syntax

`cmos [ address [ value ]]`

Description

Cmos displays and sets the values in the **CMOS** data base. The **CMOS** data base stores the configuration information about your computer. XENIX uses this information during startup to define the system hardware configuration.

You typically use the *cmos* command to change the configuration after adding new devices to your system. If you specify an *address* only, *cmos* displays the value at that address. If you specify both an *address* and a *value*, *cmos* sets the value of that address. If you omit both, *cmos displays* the entire contents of the data base.

Files

`/etc/cmos`
`/dev/cmos`

Notes

You can also alter the **CMOS** data base by writing to `/dev/cmos`

Because the system operation depends on this information being correct, only an experienced system administrator should modify the data base.

The following table describes the contents of the configuration database. For more information on the definition of the byte value of each **CMOS** location refer to the technical manual for your computer.

Table Offset	Description
00 - 0D	Real-time clock information
0E	Diagnostic status byte
0F	Shutdown status byte
10	Floppy drive type byte (Drives A and B)
11	Reserved
12	Fixed disk type byte (Drives C and D)
13	Reserved
14	Equipment byte
15	Low base memory byte
16	High base memory byte
17	Low expansion memory byte
18	High expansion memory byte
19 - 2D	Reserved
2E - 2F	CMOS checksum
30	Low expansion memory byte
31	High expansion memory byte
32	Date century byte
33	Information flags (set during power-on)
34 - 3F	Reserved

Name

console, color, monochrome, ega, keyboard – The standard system terminal.

Description

The **console** file provides access to the keyboard and the current display device of the computer and is the default device for all system error messages and for interaction during the system boot sequence. The file has the same input and output characteristics as the terminal devices described in *tty*(M).

The display normally consists of 24 lines of 80 columns each. If the display being used as the console device is changed to graphics mode, the console messages cannot be displayed. However, when you switch the console driver back to text mode, the following message appears on the console screen followed by a dump of the console message buffer:

*** Messages lost on graphics console (also check /usr/adm/messages):***

The **console** file is actually two files in one: a readable file to the keyboard and a writable file to one of three possible display devices: **color**, **monochrome**, and **ega**. The **color** file provides access to a color/graphics adapter; the **monochrome** file to the monochrome adapter; and the **ega** file to the enhanced graphics adapter. You can use only one file as the console display device at any given time. It is initially defined by the hardware configuration and assigned when the system is started. See *stty*(C) for further information about changing the file used as the console device. The other files remain available to be written to, but not read from. It is an error to attempt to access the **color**, **monochrome**, or **ega** file when no corresponding adapter exists.

Control Modes

To change the display used for the console, you can use the *stty*(C) command or you can use *ioctl*(S) with the following requests:

SWAPMONO	Selects the monochrome display as the output device for the console.
SWAPCGA	Selects the regular color display as the output device for the console.
SWAPEGA	Selects the enhanced color display as the output device for the console.

To obtain information about the display adapter type currently attached to the console, you can use *ioctl*(S) with the following request:

CONS_CURRENT

Returns the display adapter type currently attached to the console. The return value can be one of: MONO, CGA, or EGA.

Display Modes

To switch display modes on the various display devices, you can use the *stty(C)* command or you can use *ioctl(S)* with the following requests:

SW_B40x25	Selects 40x25 (40 columns x 25 rows) black and white text display mode.
SW_C40x25	Selects 40x25 color text display mode.
SW_B80x25	Selects 80x25 black and white text display mode.
SW_C80x25	Selects 80x25 color text display mode.
SW_BG320	Selects 320x200 black and white graphics display mode.
SW_CG320	Selects 320x200 color graphics display mode.
SW_BG640	Selects 640x200 black and white graphics display mode.
SW_MCAMODE	Reinitializes the monochrome adapter.

Notes

When switching from one display to another, the Start Address High and Start Address Low registers are reset to zero.

If an invalid display mode is selected, the behavior of the attached device(s) may be erratic.

The above commands are only valid for IBM or IBM compatible adapters and displays.

Getting Display Modes

To obtain information about the current display modes, you can use *ioctl(S)* with the following requests:

CGA_GET	Returns the current display mode setting of the color graphics adapter.
---------	---

EGA_GET	Returns the current display mode setting for the enhanced graphics adapter.
MCA_GET	Returns the current display mode setting for the monochrome adapter.
CONS_GET	Returns the current display mode setting for whatever display adapter is being used.

Memory Mapping Modes

The *ioctl(S)* routine can be used to map the display memory of the various devices into the user's data space. It returns a selector for the display memory. You can use the *sotofar* macro to create a far pointer from this selector so you can access the display memory. The *sotofar* macro is located in */usr/include/sys/param.h*. Refer to the hardware manuals for the details on the various displays, adapters, and controllers. You can use *ioctl(S)* requests to map the display memory as follows:

MAPCONS	Maps the display memory of the adapter currently being used as the console into the user's data space.
MAPMONO	Maps the monochrome adapter's display memory into the user's data space.
MAPCGA	Maps the color adapter's display memory into the user's data space.
MAPEGA	Maps the enhanced graphics adapter's display memory into the user's data space.

For example, you can use the following code to acquire a pointer to the start of your data space associated with the color graphics adapter display memory:

```
char far *dp;
int selector;
.
.
fd = open ("/dev/color", O_WRONLY);
selector = ioctl (fd, MAPCGA, 0);
dp = sotofar (selector, 0);
.
```

Graphics Adapter Port I/O

You can use *ioctl(S)* to input a byte from the graphics adapter port or to output a byte to the graphics adapter port. The *arg* parameter of the *ioctl* call uses the *port_io_arg* data structure:

```

struct port_io_arg {
    struct port_io_struct args[4];
};

```

As shown above, the *port_io_arg* structure points to an array of four *port_io_struct* data structures. The *port_io_struct* structure has the following format:

```

struct port_io_struct {
    char        dir;        /* direction flag (in vs.out) */
    unsigned int port;      /* port address */
    char        data;       /* byte of data */
};

```

You may specify one, two, three, or four of the *port_io_struct* structures in the array for one *ioctl* call. The value of *dir* can be either *IN_ON_PORT* to specify a byte being input from the graphics adapter port or *OUT_ON_PORT* to specify a byte being output to the graphics adapter port. *Port* is an integer specifying the port address of the desired graphics adapter port. *Data* is the byte of data being input or output as specified by the call. If you are not using any of the *port_io_struct* structures, load the *port* with 0, and leave the unused structures at the end of the array. Refer to the hardware manuals for port addresses and functions for the various adapters.

You can use the following *ioctl*(S) commands to input or output a byte on the graphics adapter port:

CGAIO Inputs or outputs a byte on the color graphics adapter port as specified.

EGAIO Inputs or outputs a byte on the enhanced graphics adapter port as specified.

MCAIO Inputs or outputs a byte on the monochrome adapter port as specified.

To input a byte on any of the graphics adapter ports, load *dir* with *IN_ON_PORT* and load *port* with the port address of the graphics adapter. The byte input from the graphics adapter port will be returned in *data*.

To output a byte, load *dir* with *OUT_ON_PORT*, load *port* with the port address of the graphics adapter, and load *data* with the byte you want output to the graphics adapter port.

Keyboard Modes

To select the mode in which keyboard characters are transmitted, use *stty*(C) or use *ioctl*(S) with the following requests:

TIOCKBOF All 7-bit ASCII characters are transmitted as usual and all 8-bit non-ASCII and extended character codes are disabled. This is the default mode.

TIOCKBON Allows 7-bit ASCII, 8-bit non-ASCII, and extended keyboard characters to be transmitted to the user program. The 7-bit and 8-bit characters are transmitted as usual. The extended characters are transmitted as a null byte followed by a second byte containing the character's extended code. When a true null byte is sent, it is transmitted as two consecutive null bytes.

8-Bit Character Codes

When the keyboard is fully enabled (TIOCKBON), an 8-bit character code can be obtained by holding down the ALT key and entering the 3-digit decimal value of the character from the numeric keypad.

Special Characters

Some keyboard characters have special meaning. For example, the ASCII DELETE (Hex 7F) has the INTERRUPT function described in *termio*(M) assigned to it. Under default operations, pressing the DELETE key generates an interrupt signal which is sent to all processes designated with the associated control terminal. When the keyboard is fully enabled (TIOCKBON), holding down the ALT key and pressing the 8 key on the home keyboard (not on the numeric keypad) returns a null byte followed by Hex 7F. This will produce the same effect as the DELETE key (Hex 7F) unless you have executed the *stty*(C) command with the *-isig* option. Refer to *stty*(C) and *termio*(M) to determine which characters have special meaning and how to control terminal input. Especially note the description of the *c_iflag*, *c_lflag*, and *c_ccarray* fields.

Function Keys

If you have not enabled the keyboard (default) and have not defined a function key using *setkey*(C), pressing the function key returns nothing. If you have defined the function, the current definition of the function key will be returned.

If you have fully enabled the keyboard (TIOCKBON) and have not defined a function key using *setkey*(C), pressing the function key will return a two-byte extended code. If you have defined the function key, the current definition of the key will be returned.

You can use the *ioctl*(S) routine to define a function key or to obtain the current definition of a function key. The *arg* parameter of the *ioctl* call uses the *fkeyarg* data structure:

```
struct fkeyarg {
    unsigned int    keynum;
    char            keydef[MAXFK]; /* Comes from ioctl.h via comcrt.h */
    char            flen;
}
```

Function keys F1 through F10 in normal case, that is a single key press, are referenced by *keynum* 1 through 10. *Keynum* 11 and 12 are reserved for future use. To program function keys F1 through F10 for double key press, such as SHIFT-Fn, CTRL-Fn, or ALT-Fn, set the *keynum* values to $12 + n$, $2 * 12 + n$, or $3 * 12 + n$. The value *n* can be an integer between 1 and 10 and represents the function key number. Values 11 and 12 are reserved for future use.

The permissible values for the function keys are:

Keynum	Function keys
0 - 11	F1 - F12
12 - 23	SHIFT F1 - SHIFT F12
24 - 35	CTRL F1 - CTRL F12
36 - 47	ALT F1 - ALT F12

You can use the following *ioctl*(S) requests to obtain or assign function key definitions:

GETFKEY	Obtains the current definition of a function key. The function key number must be passed in <i>keynum</i> (see <i>arg</i> struct above). The string currently assigned to the key will be returned in <i>keydef</i> and the length of the string will be returned in <i>flen</i> when the <i>ioctl</i> is performed.
SETFKEY	Assigns a given string to a function key. The function key number must be passed in <i>keynum</i> , the string must be passed in <i>keydef</i> , and the length of the string (number of characters) must be passed in <i>flen</i> . The <i>flen</i> argument should include the null terminator as part of the string. That is, null string will have <i>flen</i> as 1.

Control Sequences

The following is a list of the control sequences supported by the display devices. Note that "ESC" is the ASCII ESCAPE character, \033, and "Pn" is a numeric parameter to be supplied. Although spaces are shown between characters in a sequence, they are not allowed when actually forming the sequence.

ANSI Mnemonic	Sequence	Action
CNL	ESC [Pn E	Move cursor to next line
CPL	ESC [Pn F	Move cursor to previous line
CUB	ESC [Pn D	Move cursor backwards
CUF	ESC [Pn C	Move cursor forward
CUU	ESC [Pn A	Move cursor up
CUD	ESC [Pn B	Move cursor down

CUP	ESC [Pm ; Pn H	Move cursor to row Pm, column Pn
DCH	ESC [Pn P	Delete character
DL	ESC [Pn M	Delete line
ECH	ESC [Pn X	Erase character
ED	ESC [Pn J	Erase display
EL	ESC [Pn K	Erase line
ICH	ESC [Pn @	Insert char
IL	ESC [Pn L	Insert line
SD	ESC [Pn T	Scroll down
SGR	ESC [Pn m	Select graphic rendition
SU	ESC [Pn S	Scroll up

For the ED and EL sequences, if Pn is

- 0 Erase from cursor to end of the screen or line
- 1 Erase from start of screen or line to cursor
- 2 Erase entire screen or line

For the SGR sequence, if Pn is

- 0 Enter normal video mode
- 7 Enter inverse video mode
- 21 Exit inverse video mode (non-ANSI standard)

For all other sequences, Pn defines the number of times to repeat the given action. The default count is 1.

Keyboard Control Sequences

You can assign values to function keys using the *setkey*(C) command or by following the process described as follows. The special character sequence:

ESC Q Pd Pc Ps Pc

sets the output of the function key given by Pd to the string Ps. The key number Pd must be one less than the actual function key number. The string Ps must be enclosed by the quoting character Pc, and must not contain any occurrence of Pc. For example, the sequence:

ESC Q 0 "date"

sets the output of function key 1 to "date". You use the escape sequence

`^Pn`

in the string Ps to represent an ASCII control character (0x0 to 0x20). Pn must be a character in the ASCII range 0x20 to 0x40. For example, the escape sequence `“^*”` represents the newline character CTRL-J. A caret `“^”` in Ps must be given as `“^^”`.

Files

`/dev/console`
`/dev/color`
`/dev/monochrome`
`/dev/ega`
`/usr/include/sys/param.h`

See Also

`cmos(M)`, `ioctl(S)`, `stty(C)`, `termio(M)`

Notes

The CTRL- code is not available from the keyboard. The CTRL-Z code is not available from normal shells. This is due to the fact that job control has not been fully implemented under XENIX. You can get a CTRL-Z code by typing `“stty -isig”`. This also turns off all signal generation abilities from the keyboard (no DEL or CTRL-).

Name

daemon.mn – Micnet mailer daemon

Syntax

/usr/lib/mail/daemon.mn [-ex]

Description

The mailer daemon performs the “backend” networking functions of the *mail(C)*, *rcp(C)*, and *remote(C)* commands by establishing and servicing the serial communication link between computers in a Micnet network.

When invoked, the daemon creates multiple copies of itself, one copy for each serial line used in the network. Each copy opens the serial line, creates a startup message for the LOG file corresponding to that line, and waits for a response from the daemon at the other end. The startup message lists the names of the machines to be connected, the serial line to be used, and the current date and time. If the daemon receives a correct response, it establishes the serial link and adds the message “first handshake complete” to the LOG file. If there is no response the daemon waits indefinitely.

If invoked with the *-x* option, the daemon records each transmission in the LOG file. A transmission entry shows the direction of the transmission (tx for transmit, rx for receive), the number of bytes transmitted, the elapsed time for the transmission (in minutes and seconds), and the time of day of the transmission (in hours and minutes). Each entry has the form:

direction byte_count elapsed_time @ time_of_day

If invoked with the *-e* option, the daemon records all transmission errors in the LOG file. An error entry shows the cause of the error preceded by the name of the daemon subroutine that detected the error.

The mailer daemon is normally invoked by the *start* option of the *netutil* command and is stopped by the *stop* option.

During the normal course of execution, the mailer daemon uses several files in the */usr/spool/micnet/remote* directory. These files provide storage for LOG entries, commands issued by the *remote(C)* command, and a list of processes under daemon control.

Files

/usr/lib/mail/daemon.mn

/usr/spool/micnet/remote/*/LOG

/usr/spool/micnet/remote/*/mn

/usr/spool/micnet/remote/local/mn*

/usr/spool/micnet/remote/lock

/usr/spool/micnet/remote/pids

See Also

date(C), mail(C), netutil(C), rcp(C), remote(C)

Name

default – Default program information directory.

Description

The files in the `/etc/default` directory contain the default information used by system commands such as `backup(C)` and `remote(C)`. Default information is any information required by the command that is not explicitly given when the command is invoked.

The directory may contain zero or more files. Each file corresponds to one or more commands. A command searches a file whenever it has been invoked without sufficient information. Each file contains zero or more entries which define the default information. Each entry has one of the following forms:

keyword or *keyword=value*

where *keyword* identifies the type of information available and *value* defines its value. Both *keyword* and *value* must consist of letters, digits, and punctuation. The exact spelling of a *keyword* and the appropriate *values* depend on the command and are described with the individual commands.

Any line in a file beginning with a number sign (#) is considered a comment and is ignored.

Files

`/etc/default/backup`
`/etc/default/cron`
`/etc/default/dos`
`/etc/default/dumpdir`
`/etc/default/localprint`
`/etc/default/login`
`/etc/default/lpd`
`/etc/default/mkuser`
`/etc/default/passwd`
`/etc/default/micnet`
`/etc/default/restor`
`/etc/default/su`

See Also

`backup(C)`, `cron(C)`, `dos(C)`, `dumpdir(C)`, `login(C)`, `lp(C)`, `mkuser(C)`, `pwadmin(C)`, `quot(C)`, `remote(C)`, `restore(C)`, `su(C)`



Name

environ – The user environment.

Description

The user environment is a collection of information about a user, such as his login directory, mailbox, and terminal type. The environment is stored in special “environment variables,” which can be assigned character values, such as names of files, directories, and terminals. These variables are automatically made available to programs and commands invoked by the user. The commands can then use the values to access the user’s files and terminal.

The following is a short list of environment variables.

PATH	Defines the search path for the directories containing commands. The system searches these directories whenever a user types a command without giving a full pathname. The search path is one or more directory names separated by colons (:). Initially, PATH is set to <code>:/bin:/usr/bin</code> .
HOME	Names the user’s login directory. Initially, HOME is set to the login directory given in the user’s <code>passwd</code> file entry.
TERM	Defines the type of terminal being used. This information is used by commands such as <code>more(C)</code> which rely on information about the capabilities of the user’s terminal. The variable may be set to any valid terminal name (see <code>terminals(M)</code>) directly or by using the <code>tset(C)</code> command.
TZ	Defines time zone information. This information is used by <code>date(C)</code> to display the appropriate time. The variable may have any value of the form <code>xxznzz</code> where <code>xxx</code> is standard local time zone abbreviation, <code>n</code> is the difference in hours from GMT, and <code>zzz</code> is the daylight-saving local time zone abbreviation (if any). For example, <code>EST5EDT</code> . The difference for a location east of England can be given as a negative number.

The environment can be changed by assigning a new value to a variable. An assignment has the form:

name=value

For example, the assignment:

TERM=h29

sets the TERM variable to the value "h29". The new value can be "exported" to each subsequent invocation of a shell by exporting the variable with the *export* command (see *sh*(C)) or by using the *env*(C) command.

You may also add variables to the environment, but you must be sure that the new names do not conflict with exported shell variables such as MAIL, PS1, PS2, and IFS. Placing assignments in the .profile file is a useful way to change the environment automatically before a session begins.

Note that the environment is made available to all programs as an array of strings. Each string has the form:

name=value

where the *name* is the name of an exported variable and the *value* is the variable's current value. For programs started with a *exec*(S) call, the environment is available through the external pointer *environ*. For other programs, individual variables in environment are available through *getenv*(S) calls.

See Also

env(C), *exec*(S), *getenv*(S), *login*(M), *profile*(M), *sh*(C)

Name

fd0, fd1 – Floppy diskette drives

Description

Certain XENIX commands, such as **mount** , **tar** , and **backup** , use the *fd0* and *fd1* files to access the floppy diskette drives. The device *fd0* accesses the primary floppy drive, Drive 0, and *fd1* accesses the optional secondary floppy drive, Drive 1.

The devices, *fd0* and *fd1* , are actually links to 2 of the many possible device files. Each of these files corresponds to a particular type of diskette and drive:

Drive 0:

File	Density	Sectors	Sides	Capacity
fd048ss8	48 TPI	8	1	160 Kbytes
fd048ds8	48 TPI	8	2	320 Kbytes
fd048ss9	48 TPI	9	1	180 Kbytes
fd048	48 TPI	9	2	360 Kbytes
fd048ds9	48 TPI	9	2	360 Kbytes
*fd096	96 TPI	15	2	1200 Kbytes
*fd096ds15	96 TPI	15	2	1200 Kbytes

Drive 1:

File	Density	Sectors	Sides	Capacity
fd148ss8	48 TPI	8	1	160 Kbytes
fd148ds8	48 TPI	8	2	320 Kbytes
fd148ss9	48 TPI	9	1	180 Kbytes
fd148	48 TPI	9	2	360 Kbytes
+fd148ds9	48 TPI	9	2	360 Kbytes
fd196	96 TPI	15	2	1200 Kbytes
+fd196ds15	96 TPI	15	2	1200 Kbytes

* The device, *fd0* , is currently linked to these files. You can access this file through *fd0* , or directly, by specifying the appropriate device filename.

+ If you have a secondary floppy drive, the device, *fd1* , is currently linked to one of these files, depending on the type of floppy disk drive you have. You can access the appropriate file through *fd1* , or directly, by specifying the appropriate device name.

You can also access the floppy drives through a raw file (or character I/O), *rfd0* and *rfd1* . (The **format** command uses the raw device files.)

Files

Block I/O:

/dev/fd0	/dev/fd1 *
/dev/fd048ss8	/dev/fd148ss8
/dev/fd048ds8	/dev/fd148ds8
/dev/fd048ss9	/dev/fd148ss9
/dev/fd048ds9	/dev/fd148ds9
/dev/fd096ds15	/dev/fd196ds15
/dev/fd048	/dev/fd148
/dev/fd096	/dev/fd196

Character I/O:

/dev/rfd0	/dev/rfd1 *
/dev/rfd048ss8	/dev/rfd148ss8
/dev/rfd048ds8	/dev/rfd148ds8
/dev/rfd048ss9	/dev/rfd148ss9
/dev/rfd048ds9	/dev/rfd148ds9
/dev/rfd096ds15	/dev/rfd196ds15
/dev/rfd048	/dev/rfd148
/dev/rfd096	/dev/rfd196

* These files exist only if you have a secondary floppy drive.

See Also

backup(C), tar(C), mount(C), format(M)

Notes

If you did not have a second floppy drive connected when you installed XENIX, the files *fdl* and *rfdl* are not present. If you decide to add a second floppy drive later, you must use LN (C) to link *fdl* and *rfdl* to the appropriate devices.

Each file corresponds to only one floppy drive. You can access the file only if the corresponding drive is present.

Do not attempt to access a diskette or drive of one format through a device file of a different format. This gives unpredictable results and can cause errors.

When using the raw device files, you must align your input and output on 512 byte boundaries. Format handles this automatically.

The DOS utilities allow the use of aliases to access the devices files:

Alias	File
A	fd048ds9, fd048ds8, fd048ss9, fd048ss8
B	fd148ds9, fd148ds8, fd148ss9, fd148ss8,
X	fd096ds15
Y	fd196ds15

Warning

Never remove a floppy diskette from its drive while the drive light is on.



Name

fdisk – Creates the disk partition table on the fixed disks

Syntax

fdisk [-x] *special-file*

Description

Fdisk creates a disk partition table for the fixed disk specified by *special-file*. A *special file* is a device file which represents the character device of the fixed disk. A partition table defines the location, type, and size of as many as 4 partitions on any given fixed disk. Each partition acts as a unique storage area that cannot be accessed from another partition.

The -x options tells *fdisk* to remove any and all existing partitions except the Bad Track Table (BTT).

Special-file is the name of the raw device file which represents the entire fixed disk. When you execute *fdisk*, it displays its startup dialog and its prompt (*). At this time, you can enter any of *fdisk's* options.

- a Activates a partition. *Fdisk* prompts you to enter the partition's number. The operating system is always loaded from the current active partition.
- c Creates a disk partition. *Fdisk* prompts you to enter the partition's number (1, 2, 3, or 4), the starting cylinder number, the size in blocks, and the partition type. Type defines the type of file system to be stored on that partition. It can be either *system* or *user*. You can create only 4 partitions on any fixed disk. The partitions cannot overlap disk space, and cannot be larger than the available disk space. If a partition already exists, you must delete it before you can recreate it.
- d Deletes a partition. *Fdisk* prompts you to enter the partition's number.
- p Prints information on the current partition table. The table shows the partition number, its type, state, starting and ending cylinders, and its size in sectors. The partition's state is either A for active or N for not active.
- q Quits *fdisk*. Control returns to your current shell. Any changes you make are lost if you did not use the w command.
- w Writes the partition table to the fixed disk. This saves any changes you have made and overwrites any existing partition table stored on the fixed disk.
- ? Provides on-line help by displaying a list of *fdisk's* commands.

Files

```
/etc/fdisk  
/dev/rhd00  
/dev/rhd10
```

Notes

Fdisk displays a warning message if you try to write a partition table to a disk before activating a partition. *Fdisk* continues after displaying the warning.

You cannot update the partition table of an active disk. To modify the partition table of the primary fixed disk, load the XENIX Installation diskette. At the boot prompt, type:

```
fd /xenix.fd
```

and press RETURN . Once XENIX is running, you can execute *fdisk* to modify the primary fixed disk with both creation and deletion options enabled.

Name

`getty` – Sets terminal type, modes, speed, and line discipline.

Syntax

```
/etc/getty [ -h ] [ -t timeout ] line [ speed [ type [ linedisc ] ] ]
/etc/getty -c file
```

Description

Getty is a program that is invoked by *init*(M). It is the second process in the series, (*init-getty-login-shell*), that ultimately connects a user with the XENIX system. Initially *getty* prints the login message field for the entry it is using from */etc/gettydefs*. *Getty* reads the user's login name and invokes the *login*(M) command with the user's name as an argument. While reading the name, *getty* attempts to adapt the system to the speed and type of terminal being used.

Line is the name of a tty line in */etc/ttys* to which *getty* is to attach itself. *Getty* uses this string as the name of a file in the */dev* directory to open the terminal for reading and writing. Unless *getty* is invoked with the *-h* flag, *getty* will force a hangup on the line by setting the speed to zero before setting the speed to the default or specified speed. The *-t* flag, plus *timeout* in seconds, specifies that *getty* should exit if the *open* on the line succeeds and no one types anything in the specified number of seconds. The optional second argument, *speed*, is a label to a speed and tty definition in the file */etc/gettydefs*. This definition tells *getty* what speed to initially run, what the login message should look like, what the initial tty settings are, and what speed to try next should the user indicate that the speed is inappropriate (by typing a character). The default *speed* is 300 baud. The optional third argument, *type*, is a character string describing to *getty* what type of terminal is connected to the line in question. *Getty* understands the type *none*—any CRT or normal terminal unknown to the system. This is the default and only supported type.

The optional fourth argument, *linedisc*, is a character string describing which line discipline to use in communicating with the terminal. The support for line disciplines is available in the operating system but there is only one presently available, the default line discipline, *LDISC0*.

When given no optional arguments, *getty* sets the *speed* of the interface to 300 baud, specifies that raw mode will be used (awaken on every character), that *echo* will be suppressed, either parity allowed, that *NEWLINE* characters will be converted to *RETURN-LINEFEED*, and that *TAB* expansion will be performed on the standard output. It types the login message before reading the user's name a character at a time. If a null character (or framing error) is received, it is assumed to be the result of the user pushing the key. This will cause *getty* to attempt the next *speed* in the series. The series that *getty* tries is determined by what it finds in */etc/gettydefs*.

The user's name is terminated by a NEWLINE or RETURN character. The latter results in the system being set to treat RETURN appropriately (see *ioctl(S)*).

The user's name is scanned to see if it contains any lowercase alphabetic characters; if not, and if the name is given, the system is told to map any future uppercase characters into the corresponding lowercase characters.

Finally, *login* is called with the user's name as an argument. Additional arguments may be typed after the login name. These are passed to *login*, which will place them in the environment (see *login(M)*).

A check option is provided. When *getty* is invoked with the *-c* option and *file*, it scans the file as if it were scanning */etc/gettydefs* and prints out the results to the standard output. If there are any unrecognized modes or improperly constructed entries, it reports these. If the entries are correct, it prints out the values of the various flags. See *ioctl(S)* to interpret the values. Note that some values are added to the flags automatically.

Files

/etc/gettydefs
/etc/ttys

See Also

gettydefs(F), *init(M)*, *ioctl(S)*, *login(M)*, *ttys(M)*

Name

group – Format of the group file.

Description

Group contains the following information for each group:

- Group name
- Encrypted password (optional)
- Numerical group ID
- Comma-separated list of all users allowed in the group

This is an ASCII file. The fields are separated by colons; each group is separated from the next by a NEWLINE. If the password field is null, no password is demanded.

This file resides in the */etc* directory. Because of the encrypted passwords, it can and does have general read permission and can be used, for example, to map numerical group IDs to names.

Files

/etc/group

See Also

newgrp(C), *passwd(C)*, *passwd(M)*



Name

hd00, hd10 – Fixed disk drives

Description

Certain XENIX commands use the *hd00* and *hd10* files to access the fixed disk drives. The device file *hd00* accesses the primary fixed disk, and *hd10* accesses the optional secondary fixed disk. Each file allows access to the entire contents of the fixed disk, regardless of any partitions.

There are other files used to access the different partitions. They are:

Primary Fixed Disk:		Secondary Fixed Disk:	
File	Accesses	File	Accesses
hd01	First partition	hd11	First partition
hd02	Second partition	hd12	Second partition
hd03	Third partition	hd13	Third partition
hd04	Fourth partition	hd14	Fourth partition
hd0a	Active partition	hd1a	Active partition
hd0d	DOS partition	hd1d	DOS partition

You can access a partition only if it was created with *fdisk* . You also use *fdisk* to set the active partition. Only one on a fixed disk partition can be active at a time.

You can also access the fixed disk through a raw device (or character I/O) file, *rh00* and *rh10* . The *fdisk* program uses the raw device files.

Files

Block I/O:

/dev/hd00	/dev/hd10
/dev/hd01	/dev/hd11
/dev/hd02	/dev/hd12
/dev/hd03	/dev/hd13
/dev/hd04	/dev/hd14

Character I/O:

/dev/rhd00	/dev/rhd10
/dev/rhd01	/dev/rhd11
/dev/rhd02	/dev/rhd12
/dev/rhd03	/dev/rhd13
/dev/rhd04	/dev/rhd14

Notes

Each device file corresponds to only one fixed disk drive. You can access the file only if the corresponding drive is present.

When using the raw device files, you must align your input and output on 1024 byte boundaries. **Fdisk** handles this automatically.

Many fixed drives are listed by a “type”. The following is a list of drive types and their associated parameters.

Drive Type	Description	Write Pre-Comp	Landing Zone
1	306 cylinders, 4 heads	128	305
2	615 cylinders, 4 heads	300	615
3	615 cylinders, 6 heads	300	615
4	940 cylinders, 8 heads	512	940
5	940 cylinders, 6 heads	512	940
6	615 cylinders, 4 heads	no	615
7	462 cylinders, 8 heads	256	511
8	733 cylinders, 5 heads	no	733
9	900 cylinders, 15 heads	no	901
10	820 cylinders, 3 heads	no	820
11	855 cylinders, 5 heads	no	855
12	855 cylinders, 7 heads	no	855
13	306 cylinders, 8 heads	128	319
14	733 cylinders, 7 heads	no	733
15	undefined		

See Also

`fdisk(M)`

Name

`init` – Process control initialization.

Syntax

`/etc/init`

Description

The *init* program is invoked as the last step of the boot procedure and as the first step in enabling terminals for user logins. *Init* is one of three programs (*init*, *getty*(M), and *login*(M)) used to initialize a system for operation.

Init creates a process for each terminal on which a user may log in. It begins by opening the console device, `/dev/console`, for reading and writing. It then invokes a shell which asks for a password to start the system in maintenance mode. The user may type the password or terminate the shell by typing ASCII end-of-transmission (CTRL-D) at the console. If the shell terminates, *init* performs several steps to begin normal operation. It invokes a shell and reads the commands in the `/etc/rc` file. This command file performs housekeeping tasks such as removing temporary files, mounting file systems, and starting daemons. Then *init* reads the `/etc/ttys` file and forks several times to create a process for each terminal device in the file. Each line in `/etc/ttys` lists the state of the line (0 for closed, 1 for open), the line mode, and the serial line (see *ttys*(M)). Each process opens the appropriate serial line for reading and writing, assigning the file descriptors 0, 1, and 2 to the line and establishing it as the standard input, output, and error files. If the serial line is connected to a modem, the process delays opening the line until someone has dialed up and a carrier has been established on the line.

Once *init* has opened a line, it executes the *getty* program, passing the line mode as an argument. The *getty* program reads the user's name and invokes *login*(M) to complete the login process (see *getty*(M) for details). *Init* waits until the user logs out by typing ASCII end-of-transmission (CTRL-D) or by hanging up. It responds by waking up and removing the former user's login entry from the `utmp` file, which records current users, and makes a new entry in the `wtmp`(F) file, which contains a history of logins and logouts. Then the corresponding line is reopened and *getty* is reinvoked.

Init has special responses to the hangup, interrupt, and quit signals. The hangup signal SIGHUP causes *init* to change the system from normal operation to maintenance mode. The interrupt signal SIGINT causes *init* to read the `ttys` file again to open any new lines and close lines that have been removed. The quit signal, SIGQUIT, causes *init* to disallow any further logins. In general, these signals have an adverse effect on the system and should not be used by an inexperienced user. Instead, similar functions can be safely performed with the *enable*(C), *disable*(C), and *shutdown*(C) commands.

Files

/dev/tty*

/etc/utmp

/etc/wtmp

/etc/ttys

/etc/rc

/etc/gettydefs

See Also

disable(C), enable(C), getty(M), gettydefs(F), kill(C), login(M), sh(C),
shutdown(C), ttys(M)

Name

`installidd` – Installs installable device drivers.

Syntax

```
installidd [-m master ] [-c config ] [-d directory ]
```

Description

You can use the *installidd* program to produce a new configuration when installing a new hardware device if the software device driver is an “installable device driver.” The program reads two files to get information about the present hardware configuration. These files are the *config* file (default is */lib/sys/config.sys*) and the *master* file (default is */lib/sys/master*). The system administrator may use different files by specifying the *-c* option for a different *config.sys* file and/or the *-m* option for a different *master* file. The *installidd* program assumes that these two files accurately represent the current hardware configuration before the new hardware device is added.

The *installidd* program configures the new hardware device into the system by incorporating information from a new *config.sys* file that contains configuration information about the new hardware device. Later, the new device driver is linked in by the *boot* program’s *cf* option. By default, the *installidd* program assumes that the new *config.sys* file and the new device driver are on a floppy diskette in the primary floppy disk drive. With the *-d* option, the system administrator may specify a directory where these files may be found.

If there is an *init.sys* file present, it is executed as a shell script. This file could be used to create block device or character device files for the new device or to perform any other installation procedures.

Resolution of major device numbers:

The *installidd* program edits the new *config.sys* file into the present *config.sys* file without any further action necessary on the system administrator’s part unless a conflict between major numbers occurs. If the new device’s character or block device major numbers conflict with an existing device, the program warns the system administrator that the new board’s device numbers have been changed to the highest major number that is currently available. In this event, the device special files created earlier will also be recreated to reflect the change.

Resolution of interrupt vectors:

The *installidd* program requires no interaction unless there is an interrupt vector conflict. If the interrupt vector(s) that the new device uses are already claimed by existing devices, the program provides several options to the system administrator so that the conflicts may be resolved. These choices include :

Change an interrupt vector for the new device.

This option may be chosen if the new hardware device can be configured to interrupt on another (unclaimed) interrupt vector. Check the new hardware device's documentation for details on how to configure the board to do this. Some hardware devices are configured so that they must always use a certain interrupt vector. If so, this option may not be used to solve any interrupt vector conflicts.

Eliminate an existing device.

This option may be chosen if there is an existing device in the present system configuration that the system administrator does not need to have in the system when the new hardware device is installed in the system. This option may also be used to eliminate a device in the configuration which the administrator does not actually have, but was included in the configuration files for completeness. Be sure that the existing device is not a required part of the system. For example, the hard disk driver, *hd* cannot be removed from the system, but other devices such as line printers and serial devices may be removed from the system.

Delete an interrupt vector from an existing device.

This option may be used if the configuration files list a device that claims more than one interrupt vector, and the system actually only uses one of those interrupt vectors at the present time. For example, if the system only contains one serial card, the interrupt vector that is claimed for the second serial card may be deleted from the configuration files and be used for the new hardware device if necessary. It is important that the system administrator configure the devices to reflect these changes. For example, if it is necessary to remove interrupt vector 3 from the serial device, the system administrator must configure the serial card to be the first serial port, if it was previously configured to be the second serial port. For details, see the documentation for the hardware device. For standard serial/parallel boards, the following is true:

Device	Interrupt Vector
Serial Board 1	4
Serial Board 2	3
Parallel Board 1	7
Parallel Board 2	5

Delete an interrupt vector from the new device.

This option may be used if the new hardware device being installed claims more than one interrupt vector in its configuration file, but the system administrator is installing less than the maximum number of boards that may exist in the system. For example, if a new hardware device claims two interrupt vectors, and the system administrator is only installing one board of the new device, the unused interrupt vector may be deleted, resolving an interrupt conflict in the process.

After the *installidd* program has been run successfully, the system administrator should shut the system down so that a new kernel can be configured that contains the new installable device driver. When the system is rebooted, the administrator use the *cf* option at the boot prompt.

Abbreviations

The *installidd* program uses certain abbreviations to refer to the various hardware devices on the system. The more common abbreviations are:

Abbreviation	Device
hd	Hard Disk Drive
fd	Floppy Disk Drive
sio	Single Serial Adaptor (tty00 and tty01)
msa	Multichannel Board (tty02 - tty09)
lp	Line Printer
cd	Cartridge Disk Drive
mt	Cartridge Tape Drive
cons	Console

Examples

To install the disk cartridge driver, enter:

```
installidd -d /lib/sys/cd
```

To install the standard serial driver, enter:

```
installidd -d /lib/sys/sio
```

To install the parallel driver, enter:

```
installidd -d /lib/sys/lp
```

To install the tape cartridge driver, enter:

```
installidd -d /lib/sys/mt
```

To remove all installable device drivers and start over, enter:

```
rm /lib/sys/config.sys
```

Files

/etc/installidd	The installidd program
/lib/sys/master	Default master configuration file
/lib/sys/config.sys	Default installable device driver configuration file

See Also

config(CP), master(F), boot(M), xconf(CP), Chapter 10 of the *XENIX Operations Guide*

Notes

In order to change any device that appears in the master file, the Software Development System must be present on the system so that a new *c.o* file may be compiled.

The following interrupt vectors are considered valid: 3, 4, 5, 6, 7, 10, 11, 12, 13, 14, and 15.

Vectors 10-15 are assumed to interrupt on the slave interrupt controller on level 2 of the master interrupt controller.

Sharing interrupt vectors is neither supported nor recommended by this program on this machine architecture.

Name

ld – Invokes the link editor.

Syntax

ld [options] filename...

Description

Ld is the XENIX link editor. It creates an executable program by combining one or more object files and copying the executable result to the file **a.out**. The *filename* must name an object or library file. These names must have the “.o” (for object) or “.a” (for archive library) extensions. If more than one name is given, the names must be separated by one or more spaces. If errors occur while linking, *ld* displays an error message; the resulting **a.out** file is unexecutable.

Ld concatenates the contents of the given object files in the order given in the command line. Library files in the command line are examined only if there are unresolved external references encountered from previous object files. Library files must be in *ranlib*(CP) format, that is, the first member must be named `__SYMDEF`, which is a dictionary for the library. *Ld* ignores the modification dates of the library and the `__SYMDEF` entry, so if object files have been added to the library since `__SYMDEF` was created, the link may result in an “invalid object module.”

The library is searched iteratively to satisfy as many references as possible and only those routines that define unresolved external references are concatenated. Object and library files are processed at the point they are encountered in the argument list, so the order of files in the command line is important. In general, all object files should be given before library files. *Ld* sets the entry point of the resulting program to the beginning of the first routine.

There are the following options:

-C

Causes the link editor to ignore the case of symbols.

-Fnum

Sets the size of the program stack to *num* bytes. Default stack size, if not given, is 4096 bytes.

-i

Creates separate instruction and data spaces for small model programs. When the output file is executed, the program text and data areas are allocated separate physical segments. The text portion will be read-only and shared by all users executing the file.

-lx

Instructs the link editor to search a library, **libx.a**, where *x* is up to seven characters. Retained only for compatibility, this option is obsolete.

-mmapfile

Creates a link map file named *mapfile* that includes public symbols.

-Ms

Creates small model program and checks for errors, such as fixup overflow. This option is reserved for object files compiled or assembled using the small model configuration. This is the default model if no **-M** option is given.

-Mm

Creates middle model program and checks for errors. This option is reserved for object files compiled or assembled using the middle model configuration. This option implies **-i**.

-MI

Creates a large model program and checks for errors. The option is reserved for object files compiled using the large model configuration. This option implies **-i**.

-Mh

Creates huge model program and checks for errors. This option is reserved for object files compiled or assembled using the huge model configuration. This option implies **-i**.

-Me

Creates mixed model program and checks for errors. This option implies **-i**.

-n number

Truncates symbols to the length specified by *number*.

-o name

Sets the executable program filename to *name* instead of **a.out**.

-P

Disables packing of segments.

-s

Strips the symbol table.

-Snum

Sets the maximum number of data segments to *num*. If no argument is given, the default is 128.

-u symbol

Designates the specified *symbol* as undefined.

Ld should be invoked using the *cc*(CP) instead of invoking it directly. *Cc* invokes *ld* as the last step of compilation, providing all the necessary C-language support routines. Invoking *ld* directly is not recommended since failure to give command line arguments in the correct order can result in errors.

Files

/bin/ld

See Also

ar(CP), cc(CP), masm(CP), ranlib(CP)

XENIX C User's Guide

Notes

The user must make sure that the most recent library versions have been processed with *ranlib*(CP) before linking. If this is not done, *ld* cannot create executable programs using these libraries.

